30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 5

Kotlin Flow & Reactive Programming — Cold vs Hot, StateFlow, Operators & Backpressure



Esmaeeil Moradi
@esmaeeil-moradi700

Why Flow is Day 5

Day 4 was about one result at a time. Real apps deal with values **over time** — database changes, sockets, location, user input. In 2026 that means `Flow`:

- Room, DataStore and Paging return `Flow`; Compose collects it with `collectAsStateWithLifecycle()`
- UI state is a `StateFlow` (Day 3) — but picking between StateFlow, SharedFlow and Channel is where bugs hide
- Sharing strategies (`stateIn`, `shareIn`) decide what keeps running in the background
- Backpressure and cancellation come for free from coroutines — if you understand them
- Many teams are still migrating from RxJava

Level	Questions
Junior	Flow vs List vs Sequence · Cold flows & collect · Operators
Mid	StateFlow vs SharedFlow vs Channel · stateIn & shareIn · Flattening · flowOn
Senior	Backpressure · callbackFlow · Lifecycle-aware collection
Architect	Errors & retry · Testing with Turbine · Migrating from RxJava



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: cold vs hot

```
COLD  flow { … } ─▶ map ─▶ filter ─▶ collect
      the producer runs once PER collector

HOT   MutableStateFlow ──┬─▶ collector A
      lives on its own   ├─▶ collector B
                         └─▶ collector C
```

- **Cold:** nothing runs until a terminal operator (`collect`, `first`, `toList`); each collector gets its own run
- **Hot:** one shared source that exists without collectors; collectors only observe it
- **`stateIn` / `shareIn`** turn a cold flow into a hot one — and you decide when it starts and stops

THREE QUESTIONS FOR EVERY STREAM

Who starts it? Who stops it? What does a late collector see? Answer those and most Flow bugs disappear.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 5



Junior

Flow vs List vs Sequence · Cold flows & collect · Operators



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Flow vs List vs Sequence

	`List`	`Sequence`	`Flow`
Evaluation	Eager — all at once	Lazy, item by item	Lazy, runs on `collect`
Waiting for the next item	—	Blocks the thread	Suspends
Values over time (DB, socket)	No	No	Yes
Cancellation	—	—	Cooperative, structured

```
interface ArticleRepository {
    suspend fun refresh()                          // one value
    fun observeArticles(): Flow<List<Article>>     // over time
}
```

One result → `suspend fun`. Values over time → `Flow`.

INTERVIEW TRAP

"A Flow can only be collected once." — Last year's version said so. A cold flow can be collected any number of times; every collection **re-runs** the producer.

INTERVIEW TRAP

`flow { emit(api.getUser()) }` for a one-shot call — a `suspend fun` in disguise: harder to call, harder to test.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Cold flows and terminal operators

```
val ticks: Flow<Int> = flow {
    println("start")            // runs for EVERY collector
    repeat(3) { i ->
        delay(100)
        emit(i)
    }
}

viewModelScope.launch {
    ticks.collect { println(it) }   // start 0 1 2
    ticks.collect { println(it) }   // start 0 1 2 — again
    println("done")                 // after both complete
}
```

- `flow { }` only **describes** the work; terminal operators run it: `collect`, `first()`, `toList()`, `single()`, `launchIn(scope)`
- Intermediate operators (`map`, `filter`…) are lazy too
- A flow is **sequential**: `emit` suspends until the collector has processed the value



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. What interviewers check

```
lifecycleScope.launch {
    viewModel.state.collect { render(it) }
    viewModel.events.collect { handle(it) }  // ❌ never reached
}
```

INTERVIEW TRAP

"The next line runs once `collect` has started." — `collect` suspends until the flow **completes**. Room, `StateFlow` and sockets never complete, so code after it never runs. Launch each endless collection in its own coroutine.

INTERVIEW TRAP

"Never call `collect` on the main thread." — Last year's version said so. Collecting on Main is normal: `collect` suspends, it doesn't block. What matters is where the **upstream work** runs (Q7).

FOLLOW-UP

"`launchIn(scope)` vs `collect`?" — `flow.onEach { … }.launchIn(scope)` is shorthand for `scope.launch { flow.collect { … } }` and returns the `Job`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Everyday operators

Operator	Does	Typical use
`map`, `filter`, `onEach`	Transform, drop, side effect per item	DTO → UI model, logging
`transform`	Emits zero or more values per item	Expand or skip with logic
`distinctUntilChanged`	Skips consecutive duplicates	Avoid redundant work
`debounce(ms)`	Emits after a quiet period	Search box input
`sample(ms)`	Latest value once per period	Fast sensors, progress
`take(n)`, `first()`	Stops after n items	One-off reads

INTERVIEW TRAP

"Use `debounce` to smooth a fast price stream." — Last year's version suggested it. `debounce` only emits after a **quiet** period: if updates keep arriving faster than the timeout, it emits nothing. Use `sample` or `conflate`.

INTERVIEW TRAP

"Flow has `throttleFirst` and `throttleLatest`." — Not in kotlinx.coroutines, although last year's version listed them. The built-ins are `debounce` and `sample`; anything else you write yourself.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. combine vs zip vs merge

```
val uiState = combine(
    repo.cart,                          // Flow<Cart>
    repo.user,                          // Flow<User>
    promo.onStart { emit(null) },       // Flow<String?>
) { cart, user, code ->
    CheckoutUiState(cart, user.address, code)
}
```

	Emits when	Completes when
`combine`	Any source emits — once all have emitted	All sources complete
`zip`	Every source has a new item — pairs them 1:1	The shortest completes
`merge`	Any source emits — same type, no pairing	All sources complete

INTERVIEW TRAP

`combine` waits until **every** source has emitted once. One silent source and the screen shows nothing. Give optional sources a start value: `onStart { emit(null) }`.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 5

Mid-Level

StateFlow vs SharedFlow vs Channel · stateIn & shareIn · Flattening · flowOn



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. StateFlow vs SharedFlow vs Channel

	`StateFlow`	`SharedFlow`	`Channel`
Holds a value	Always — `.value`	Optional replay cache	No
Initial value	Required	None	None
Equal values	Skipped (conflated)	Delivered	Delivered
Several collectors	All see the latest	All get every emission	Each item goes to **one** receiver
Nobody collecting	Keeps the latest	Drops it (unless replay / buffer)	Buffers up to its capacity, then `send` suspends
Use for	UI state	Broadcasting to many listeners	A work queue with one consumer

`StateFlow` and `SharedFlow` **never complete** — `collect` on them never returns. One-off UI events were Day 3: model them as state.



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Hot-stream traps

```
val items = mutableListOf<Item>()
_state.value = items        // emitted
items += newItem
_state.value = items        // same instance: NOT emitted
```

INTERVIEW TRAP

"StateFlow delivers every value." — It keeps only the **latest**, and skips values `equal` to the current one. A list mutated in place is invisible to collectors — and to Compose. Keep state immutable.

INTERVIEW TRAP

"SharedFlow needs an initial value and keeps the latest one." — That's `StateFlow`; last year's version described both the same way. `MutableSharedFlow()` has no value and no replay, and drops emissions when nobody collects.

INTERVIEW TRAP

`tryEmit` on a buffer-less `MutableSharedFlow()` returns `false` whenever someone is collecting — the event is silently lost. It needs `extraBufferCapacity` (or use `emit`).



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. stateIn and shareIn — cold to hot

SharingStarted	Upstream starts	Upstream stops
`Eagerly`	Immediately	Never — until the scope ends
`Lazily`	At the first collector	Never — until the scope ends
`WhileSubscribed(5_000)`	At the first collector	5 s after the last one leaves

- `stateIn` → a `StateFlow` (needs an initial value); `shareIn` → a `SharedFlow` with `replay` of N items
- `WhileSubscribed(5_000)`: a rotation doesn't restart the upstream, leaving the app stops it — the Day 3 default for UI state

INTERVIEW TRAP

`Eagerly` in a ViewModel: the work starts before anyone looks, and keeps running while the app is in the background, until the ViewModel is cleared.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Sharing one connection

```
class PriceRepository(
    socket: PriceSocket,
    appScope: CoroutineScope,
) {
    // One socket connection, shared by every screen
    val prices: SharedFlow<Price> = socket.prices()
        .shareIn(
            scope = appScope,
            started = SharingStarted.WhileSubscribed(
                stopTimeoutMillis = 5_000,
                replayExpirationMillis = 0,
            ),
            replay = 1,
        )
}
```

`replayExpirationMillis = 0` drops the cached price when the socket stops — a returning screen never flashes a stale price.

INTERVIEW TRAP

`fun prices() = socket.prices().shareIn(appScope, …)` — every **call** creates a new shared flow and a new connection. Share once, in a property.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Flattening: a search pipeline

```
val results: StateFlow<SearchState> = query
    .debounce(300)
    .map { it.trim() }
    .distinctUntilChanged()
    .flatMapLatest { q ->
        if (q.isEmpty()) flowOf(SearchState.Idle)
        else repo.search(q)
            .map { SearchState.Hits(it) }
            .onStart<SearchState> { emit(SearchState.Loading) }
            .catch { emit(SearchState.Error) }
    }
    .stateIn(
        viewModelScope,
        SharingStarted.WhileSubscribed(5_000),
        SearchState.Idle,
    )
```

A new query **cancels** the search in flight — stale results can never overwrite fresh ones.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Choosing the flatMap

Operator	Inner flows	Use for
`flatMapLatest`	A new value **cancels** the previous inner flow	Search, "selected id → details"
`flatMapConcat`	One at a time, in order	Ordered, sequential work
`flatMapMerge`	Several at once (16 by default), order not kept	Independent parallel work

INTERVIEW TRAP

`flatMapMerge` for search: a slow response for "ko" can arrive after the one for "kotlin" and overwrite it.

INTERVIEW TRAP

`catch` placed **after** `flatMapLatest`: the first error completes the whole pipeline — later queries are never processed. Catch **inside** the inner flow, as in the code.

FOLLOW-UP

"*`mapLatest` or `flatMapLatest`?*" — `mapLatest` when each value maps to **one** suspend call; `flatMapLatest` when it maps to a stream.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Where does each operator run?

```
fun logLines(file: File): Flow<LogLine> = flow {
    file.useLines { lines ->
        lines.forEach { emit(it) }      // blocking reads
    }
}
    .flowOn(Dispatchers.IO)             // ↑ runs on IO
    .map { parse(it) }                  // CPU work
    .flowOn(Dispatchers.Default)        // ↑ map runs on Default
```

- `flowOn` changes the context of everything **above** it, up to the previous `flowOn`
- The collector never moves: `collect { }` runs where it was called (e.g. Main)
- A dispatcher change inserts a **buffer** between the two sides, so they run concurrently

INTERVIEW TRAP

`withContext(Dispatchers.IO) { emit(x) }` inside `flow { }` breaks **context preservation** and throws `IllegalStateException: Flow invariant is violated`. Use `flowOn` — or `channelFlow` to emit from several coroutines.

INTERVIEW TRAP

"*`flowOn` moves the collector to another thread.*" — It only affects **upstream**. Everything after it, including `collect`, stays in the collector's context.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 5



Senior

Backpressure · callbackFlow · Lifecycle-aware collection



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Backpressure: fast producer, slow collector

By default there's no queue: `emit` suspends until the collector has processed the value, so the producer simply slows down.

```
val numbers = flow {
    for (i in 1..3) { delay(100); emit(i) }   // 100 ms each
}

numbers.collect { delay(300) }              // ≈ 1.2 s: 1 2 3
numbers.buffer().collect { delay(300) }     // ≈ 1.0 s: 1 2 3
numbers.conflate().collect { delay(300) } // gets 1 3
numbers.collectLatest { delay(300) }        // only 3 finishes
```

- `buffer` lets the producer run ahead — same values, less total time
- `conflate` drops values the collector was too busy to see
- `collectLatest` cancels the collector's unfinished work when a newer value arrives



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Choosing a strategy

Strategy	Loses	Use for
Default (none)	Nothing — the producer waits	Most flows
`buffer(n)`	Nothing — suspends when full	Two slow, independent stages
`conflate()`	Intermediate values	Prices, location, progress
`collectLatest { }`	Unfinished collector work	Expensive rendering, diffing
`buffer(n, DROP_OLDEST)`	The oldest queued values	Telemetry, logs

INTERVIEW TRAP

"*`buffer()` can grow until `OutOfMemoryError`*." — Last year's version said so. The default buffer is bounded (64 items) and suspends the producer when full; only `Channel.UNLIMITED` grows without limit.

INTERVIEW TRAP

"Flow is pull-based, like Reactive Streams." — There is no `request(n)`. Flow's backpressure is plain **suspension**: `emit` can't return until the value has been handed over.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. callbackFlow: wrapping listener APIs

```
fun ConnectivityManager.isOnline(): Flow<Boolean> =
    callbackFlow {
        val callback = object : NetworkCallback() {
            override fun onAvailable(network: Network) {
                trySend(true)
            }
            override fun onLost(network: Network) {
                trySend(false)
            }
        }
        trySend(activeNetwork != null)       // initial value
        registerDefaultNetworkCallback(callback)
        awaitClose { unregisterNetworkCallback(callback) }
    }.distinctUntilChanged()
```

- Callbacks aren't suspend functions → `trySend`. The channel buffers 64 items by default; add `conflate()` or `buffer()` to choose what happens when it's full
- `awaitClose { }` keeps the flow open and unregisters the listener when collection stops
- One value only? `suspendCancellableCoroutine` (Day 4)

INTERVIEW TRAP

Forgetting `awaitClose`: the flow throws `IllegalStateException` as soon as the block ends — and a listener that's never unregistered leaks.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Lifecycle-aware collection

Collecting with	While the app is in the background
`lifecycleScope.launch { flow.collect {} }`	❌ Keeps collecting until DESTROYED
`launchWhenStarted { }` — deprecated	❌ Collector paused, upstream still running
`repeatOnLifecycle(STARTED)`, `flowWithLifecycle`	✅ Cancelled on STOP, restarted on START
`collectAsState()` in Compose	⚠️ Collects while composed — ignores the lifecycle
`collectAsStateWithLifecycle()`	✅ Compose + lifecycle

INTERVIEW TRAP

Last year's version recommended `collectAsState` and `launchWhenStarted`. In 2026: `collectAsStateWithLifecycle()` in Compose, `repeatOnLifecycle` in Views.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. In code

```
// Compose
val state by viewModel.uiState.collectAsStateWithLifecycle()

// Views (Fragment)
viewLifecycleOwner.lifecycleScope.launch {
    viewLifecycleOwner.repeatOnLifecycle(STARTED) {
        launch { viewModel.uiState.collect(::render) }
        launch { viewModel.banner.collect(::showBanner) }
    }
}
```

- Two flows → two `launch` blocks inside `repeatOnLifecycle` (Q2)
- In Fragments use `viewLifecycleOwner`, not the Fragment itself (Day 2)

INTERVIEW TRAP

`repeatOnLifecycle` restarts collection on every START. Collect a **cold** flow that calls the network, and every return to the app repeats the request. Expose it as `stateIn(…, WhileSubscribed(5_000), …)` from the ViewModel instead.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 5

Architect

Errors & retry · Testing with Turbine · Migrating from RxJava



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Resilient streams

```
fun livePrices(): Flow<PriceState> = socket.prices()
    .map<Price, PriceState> { PriceState.Live(it) }
    .retryWhen { cause, attempt ->
        val retry = cause is IOException && attempt < 5
        if (retry) {
            emit(PriceState.Reconnecting)
            delay(1_000L shl attempt.toInt())  // 1 s … 16 s
        }
        retry
    }
    .catch { e ->
        if (e !is IOException) throw e
        emit(PriceState.Failed)
    }
```

- `retryWhen` re-collects the upstream — here, a new socket connection — up to 5 times with exponential backoff, emitting a "reconnecting" state
- `catch` turns the final **network** failure into a value the UI can render
- Anything else is a bug: it's rethrown and reaches crash reporting (Day 4, Q11)

FOLLOW-UP

"*`retry` or `retryWhen`?*" — `retry(3) { it is IOException }` for simple cases; `retryWhen` when you need the attempt number or want to emit while waiting.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. The rules of flow errors

- An exception **completes** the flow. After `catch` runs, no more upstream values arrive.
- `catch` sees only **upstream** exceptions; an exception thrown in `collect { }` passes through it.
- **Exception transparency:** never wrap `emit` in `try/catch` — it would swallow the collector's exceptions.

```
// ❌ Swallows exceptions thrown by the collector
flow {
    try { emit(value) } catch (e: Exception) { log(e) }
}
```

kotlinx.coroutines detects this: the next `emit` throws *"Flow exception transparency is violated"*.

INTERVIEW TRAP

"Put `catch` at the end so the stream stays alive." — `catch` can't resume a finished flow. For long-lived streams, `retry` the source or catch **inside** `flatMapLatest` (Q6).



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Testing flows with Turbine

```
@Test fun refreshShowsSpinnerThenPosts() = runTest {
    val repo = FakeFeedRepository(posts, delay = 1.seconds)
    val viewModel = FeedViewModel(repo)

    viewModel.uiState.test {
        assertEquals(FeedUiState(), awaitItem())
        viewModel.refresh()
        assertTrue(awaitItem().isRefreshing)
        assertEquals(posts, awaitItem().posts)
    }
}
```

- `test { }` collects the flow: `awaitItem()`, `awaitComplete()`, `awaitError()`, `expectNoEvents()`
- It fails if events are left unconsumed — tests stay exact
- Several flows at once: `turbineScope { }` with `testIn(backgroundScope)`

INTERVIEW TRAP

Remove the fake's `delay` and the test can fail: the `StateFlow` goes from "refreshing" to "done" before the collector runs, and conflation skips the spinner state. Make fakes suspend — or assert only the final state.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. From RxJava to Flow

RxJava	Coroutines / Flow
`Single<T>`, `Completable`, `Maybe<T>`	`suspend fun` returning `T`, `Unit`, `T?`
`Observable<T>`, `Flowable<T>`	`Flow<T>`
`BehaviorSubject`	`MutableStateFlow`
`PublishSubject`	`MutableSharedFlow`
`switchMap`, `concatMap`, `flatMap`	`flatMapLatest`, `flatMapConcat`, `flatMapMerge`
`subscribeOn`	`flowOn`
`observeOn`	The collector's context
`CompositeDisposable`	A scope, e.g. `viewModelScope`

INTERVIEW TRAP

"*`flowOn` is Flow's `observeOn`.*" — Opposite: `flowOn` affects **upstream**, like `subscribeOn` (but its position matters). The `observeOn` role is played by the context you collect in.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Migrating without a rewrite

```
// kotlinx-coroutines-rx3 adapters, both directions
val prices: Flow<Price> = legacyApi.priceStream().asFlow()

suspend fun loadUser(id: String): User =
    legacyApi.user(id).await()            // Single → suspend

fun cartUpdates(): Observable<Cart> =
    cartRepository.cart.asObservable()    // for Rx callers
```

- **Edges inward:** new features in Flow; adapters at the boundary; convert the data layer one repository at a time
- Keep both sides tested: Turbine for Flow, `TestObserver` for Rx
- Delete the adapters when the last Rx caller is gone

MY TAKE

The real cost of RxJava in 2026 isn't missing features — it's running **two concurrency models** in one codebase. Plan the migration as a strangler, not a big-bang rewrite.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 5 · ARCHITECT

Day 5 — cheat sheet

- One value → `suspend fun`; values over time → `Flow`. A cold flow re-runs for **every** collector.
- `collect` suspends until the flow completes — hot flows never do, so launch each collection separately.
- `debounce` for input, `sample` / `conflate` for fast streams; `combine` waits for every source once.
- `StateFlow` = state (latest, equality-conflated); `SharedFlow` = broadcast; `Channel` = one consumer.
- Share with `stateIn` / `shareIn` in a property — `WhileSubscribed(5_000)` for UI.
- `flatMapLatest` for search; catch **inside** the inner flow.
- `flowOn` affects upstream only; never `withContext` around `emit`.
- Backpressure is suspension: `buffer`, `conflate`, `collectLatest` — the default buffer is bounded.
- `callbackFlow` + `trySend` + `awaitClose`; collect with `collectAsStateWithLifecycle` / `repeatOnLifecycle`.
- Errors complete the flow: `retryWhen` with backoff, `catch` upstream only. Test with Turbine.

Day 6 is next → Jetpack essentials: ViewModel, Room, DataStore & Paging. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700